APPLICATION NOTE:
PCIE HOST NIC API FOR TILE-GX

RELEASE 1.00
DOC. NO. AN053
JANUARY 2013
TILERA CORPORATION

Publishing Information:

Document Number	AN053
Release	1.00
Date	2/1/13

Contact Information:

Tilera Corporation
Information info@tilera.com **Web Site http://www.tilera.com**

CONTENTS

PCIE HOST NIC API FOR TILE-GX

1 Overview

The TILE-Gx PCIe host-NIC interfaces provide a means of transferring data between the application's *user space* on the TILE-Gx card and the x86 host's *kernel space*, unlike other TILE-Gx PCIe programming interfaces which are designed to move data between the user spaces on the TILE-Gx device and the host.

This application note explains the use of the host-NIC API and demonstrates how you can use the API to implement a high-performance PCIe data transmission framework in your applications.

2 Development Environment Requirements

The host-NIC API is supported on the TILEncore-Gx PCIe card and x86 host platforms that are supported by the Tilera MDE 4.x releases.

3 Description of the Application

If your application requires low-latency and high throughput for data transfers between the Tile user space and the host kernel, you can build that application on top of the host-NIC API. Two typical applications are:

- a customized host packet handler
- a software-defined NIC

In a custom packet handler application, a host kernel module can be created to process packets to and from the host-NIC interfaces, at the same level as and replacing the IP layer in the Linux networking stack. The host-NIC interfaces are abstracted using the Linux kernel `net_device` structure, hence the name host-NIC interface. By itself, the host-NIC

interface cannot function as a conventional NIC. It simply provides a raw data transfer channel between the TILE-Gx user space and the host kernel space. In networking terms, it provides a point-to-point connection and is not capable of packet switching.

The so-called smart NIC application implements not only a basic Ethernet-aware PCIe NIC, i.e. interfacing to the host kernel IP stack, but potentially it can provide advanced networking off-loading features.

4 Software Architecture of the Host-NIC API

The host PCIe driver's host-NIC component (hereafter referred to as host-NIC driver) abstracts each host-NIC interface using the Linux kernel `net_device` structure, enabling the same kernel/driver API to the host-NIC interface as regular Ethernet NIC devices. The host-NIC driver implements performance-enhancing features such as PCI MSI-X interrupts and the NAPI interrupt mitigation scheme, etc.

The Tilera MDE includes a kernel packet handler which functions as an echo server, returning every ingress packet that arrives from the TILE-Gx back to the TILE-Gx. In the following sections, we use the example module located at:

```
$TILERA_ROOT/examples/trio/gxpci_host_nic/gxpci_echo_drv_module.c.
```

Using this example code, this document describes how to access the host-NIC interfaces, focusing on these operations:

- Installing a custom packet handler.
- Opening a host-NIC interface.
- Processing an ingress packet.
- Closing the host-NIC interface.
- Un-installing the custom packet handler.

4.1 Using the Host-side API

The `gxpci_echo_drv_module.c` example uses the Host-side API to perform the following operations:

1. Installing a Custom Packet Handler.

   The Linux kernel structure `packet_type` defines a packet or protocol handler. The simple echo server specifies the following two fields:

   ```
   static struct packet_type gxpci_pt __read_mostly = {
           .type = 0xABCD,
           .func = gxpci_echo_rcv,
   };
   ```

   Here, the `type` field provides the artificial packet type, similar to the `EtherType` field in an Ethernet frame. To make the Linux net device layer distribute the TILE-Gx packets to the custom packet handler, as opposed to the IP layer, we use the two-octet 0xABCD to identify the TILE-Gx packet type. The `func` field provides the pointer to the function that processes the ingress packets.

To install the packet handler, call this function:

```
dev_add_pack(&gxpci_pt);
```

2. Opening a Host-NIC Interface

   The host-NIC interfaces has device names of format `gxpci`*`N`*`-`*`M`*, where *`N`* is a positive integer representing the TILE-Gx endpoint port's link index and *`M`* is the interface index on this port in the range [0, 3]. For example, `gxpci1-0` is NIC interface #0 on TILE-Gx port #1.

   Each host-NIC interface can have a configurable number of receive and transmit queues, similar to the multi-queue design on the modern 10G NIC cards. One Receiver (RX) and Transmitter (TX) queue each is mapped to one TILE-Gx Push/Pull DMA ring, respectively. By default, each host-NIC interface has one pair of RX/TX queues. Interface queue configuration is covered in Section 6 Different Configurations and Options.

   Opening a particular host-NIC interface involves the following two steps:

   a. Retrieving the `net_device` structure that is associated with this interface, which is registered by the `host_NIC` driver when the TILE-Gx PCIe port is probed.

   b. Calling the `dev_open()` kernel function to open this interface.

```
struct net_device *net_dev;
char ifname[IFNAMSIZ];

sprintf(ifname, "gxpci%d-%lu", link_index, if_num);

net_dev = dev_get_by_name(&init_net, ifname);
err = dev_open(net_dev);
```

   `link_index` is the link index of the TILE-Gx PCIe end-point board.

   `if_num` is the interface number, conveyed as an `unsigned long`.

   `inet_name` is the applicable net name-space.

   `ifname` is the interface name.

3. Processing an Ingress Packet

   The function defined in the `packet_type` structure will be called for each ingress packet that contains the same packet type (e.g. 0xABCD). In our simple echo server handler, we simply call the kernel function `dev_queue_xmit()` to transmit the ingress packet out of the incoming host-NIC interface.

```
/*
* This is the packet handler that gets called by the network
* ingress layer to process the PCIe packets.
*/
static int gxpci_echo_rcv(struct sk_buff *skb, struct net_device
*ifp,
       struct packet_type *pt, struct net_device *orig_dev)
{
dev_queue_xmit(skb);
return 0;
}
```

At this point, you can use the `ifconfig` command to view the host-NIC interface status, as shown in the following example:

```
gxpci1-0  Link encap:Ethernet  HWaddr 32:59:8A:E5:DF:DF
          inet6 addr: fe80::3059:8aff:fee5:dfdf/64 Scope:Link
          UP BROADCAST RUNNING NOARP  MTU:4096  Metric:1
          RX packets:295701 errors:0 dropped:0 overruns:0 frame:0
          TX packets:295652 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:397422144 (379.0 MiB)  TX bytes:397356288 (378.9 MiB)
          Interrupt:90
```

Here, the Ethernet MAC address is set randomly by the host-NIC driver and is not used by the interface.

4. Closing a Host-NIC Interface

   To close a host-NIC interface, call the kernel function `dev_close()`, for example:

   ```
   dev_close(netdev);
   ```

5. Un-installing the Custom Packet Handler

   To close a custom packet handler, call the kernel function `dev_remove_pack()`, for example:

   ```
   dev_remove_pack(&gxpci_pt);
   ```

4.2 Using the Tile-side API

The tile-side host-NIC API is part of the GXPCI API outlined in the MDE document *Application Libraries Reference Manual* (UG527).

Applications can use this API to create multiple logical PCIe channels that transport packets between the TILE-Gx user space and the x86 host kernel space.

It provides functions that let you do the following:

1. Create a GXIO TRIO and a GXPCI Context

   The GXIO TRIO and GXPCI contexts provide abstraction and encapsulation for the data transfer engine implementation utilizing the TILE-Gx PCIe hardware. The user application provides the storage or memory for these two contexts and treats them as opaque objects.

   For example:

   ```
   gxio_trio_context_t trio_context_body;
   gxio_trio_context_t* trio_context = &trio_context_body;
   gxpci_context_t gxpci_context_body;
   gxpci_context_t* gxpci_context = &gxpci_context_body;

   result = gxio_trio_init(trio_context, trio_index);
   result = gxpci_init(trio_context, gxpci_context, trio_index,
   loc_mac);
   ```

   For detailed API function descriptions, refer to *Application Libraries Reference Manual* (UG527).

2. Open a Host-NIC Interface

The TILE-Gx host-NIC application must open at least two unidirectional queues, i.e. Receiver (tile-to-host) and Transmitter (host-to-tile), as transport for packet transfers to and from a host-NIC interface. The queue configuration must match the host side. See Section 6 for details.

To open a host-NIC interface, the Gx application must specify the interface index and the queue index. The host-NIC API uses an unsigned integer to represent the interface, with the low 16-bit indicating the interface index and the upper 16-bit indicating the queue index. For example, to open a tile-to-host queue, invoke the following function:

```
int asid = GXIO_ASID_NULL;
int queue_index = q_index << GXPCI_QUEUE_INDEX_QUEUE_SHIFT |
nic_index;
result = gxpci_open_queue(gxpci_context, asid, GXPCI_NIC_T2H, 0,
queue_index, 0, 0);
```

The argument `asid` indicates the Address Space ID, a term that is borrowed from the OS. It is used by the TILE-Gx PCIe hardware to set up the DMA access to the application memory space from the TILE-Gx endpoint controller. For a stand-alone host-NIC application, GXIO_ASID_NULL is used as a hint to the API that a new ASID should be allocated inside the API. In the common case of a joint mPIPE and PCIe application, you can allocate a single Address Space ID explicitly using the GXIO routine `gxio_trio_alloc_asids()` and share that ASID between the two parts of the application.

3. Allocate and Register the Data Buffer Pool

   Packet buffers that are accessed directly by the PCIe hardware must be registered with the hardware which creates I/O TLB entries to facilitate zero-copy data transfer between the TILE-Gx memory and the PCIe link. The following shows how to allocate a huge page and register it with the hardware:

   ```
   tmc_alloc_t alloc = TMC_ALLOC_INIT;
   tmc_alloc_set_huge(&alloc);
   void* buf_mem = tmc_alloc_map(&alloc, MAP_LENGTH);
   assert(buf_mem);

   result = gxpci_iomem_register(gxpci_context, buf_mem, MAP_LENGTH);
   ```

   Note: You must call the `gxpci_iomem_register()` function after you invoke the `gxpci_open_queue()` routine.

4. Conduct Packet Transfer To and From the Host

   The TILE-Gx host-NIC application should run in a loop. Every time the application enters the loop, it should perform the following operations:

   a. Determine the number of available command credit which is the maximum number of send or receive buffers that can be posted to the queue. Note that the application needs to monitor if the queue has been reset by the host side calling `dev_close()` by checking for the `GXPCI_ERESET` value.

```
  while (!done)
{
  int credits = gxpci_get_cmd_credits(context);
  if (credits == GXPCI_ERESET)
  {
    printf("Tile: do_send: channel is reset\n");
    done=1;
    break;
}
...
}
```

b. Post as many commands as possible by invoking functions `gxpci_post_cmd()`. For example”

```
// Post the next 'count' buffers.
    for (int i = 0; i < count; i++)
    {
      cmd[i].buffer = buf_mem +
        ((send_seq_num + i) % PKTS_IN_POOL) * MAX_PKT_SIZE;

      result = gxpci_post_cmd(context, &cmd[i]);
      if (result == GXPCI_ERESET)
      {
        printf("do_send: channel is reset\n");
        goto send_reset;
      }
      VERIFY_ZERO(result, "t2h: gxpci_post_cmd()");
    }
    send_seq_num += count;
}
```

c. Check if there are any completed commands by invoking function `gxpci_get_comps()`. Since this function could block, you should always check for `GXPCI_ERESET` which is returned when the function is unblocked due to channel reset.

```
result = gxpci_get_comps(gxpci_context, comp, 0, MAX_CMDS_BATCH);
  if (result == GXPCI_ERESET)
  {
    printf("Tile: do_send %d: channel is reset\n");
    done = 1;
    break;
  }
```

5 Configuring and Running the Application and Test Equipment

Using a TILE-Gx PCIe card on a x86 host, you can run an example program demonstrating the API. For information about running the example program, refer to the `README.txt` file in:

```
$TILERA_ROOT/examples/trio/gxpci_host_nic
```

6 Different Configurations and Options

By default, 4 software-defined NIC interfaces (`gxpci`*`X`*`-0|1|2|3`) are created for each TILE-Gx endpoint port on the x86 host and each interface has one receiver (RX) and one transmitter (TX) queue. To change the default configuration, use the host driver arguments on the host side and kernel arguments on the TILE side.

1. Change the number of virtual NIC interfaces per TILE-Gx port
   - Host side: host driver argument `nic_ports`.
   - TILE side: kernel argument `pcie_host_vnic_ports=`*`T`*`,`*`P`*`,`*`N_nic`*, where *`T`* is the TRIO instance number, *`P`* is the port number and *`N_nic`* is the number of NIC interfaces.
2. Change the number of the TX queues per NIC interface
   - Host side: host driver argument `nic_tx_queues`.
   - TILE side: kernel argument `pcie_host_vnic_tx_queues=`*`T`*`,`*`P`*`,`*`N_tq`*, where *`T`* is the TRIO instance number, *`P`* is the port number and *`N_tq`* is the number of TX queues.
3. Change the number of the RX queues per NIC interface
   - Host side: host driver argument `nic_rx_queues`.
   - TILE side: kernel argument `pcie_host_vnic_rx_queues=`*`T`*`,`*`P`*`,`*`N_rq`*, where *`T`* is the TRIO instance number, *`P`* is the port number and *`N_rq`* is the number of RX queues.

Given these interdependencies, the following commands configure the Host and Tile sides to support 3 software-defined NIC interfaces and 2 receiver and transmitter queues per interface:

Host side:

```
$TILERA_ROOT/lib/modules/tilepci-install \
nic_ports=3 nic_tx_queues=2 nic_rx_queues=2
```

Tile side:

```
tile-monitor --hvx pcie_host_nic_ports=0,0,3 \
--hvx pcie_host_vnic_rx_queues=0,0,2 \
--hvx pcie_host_vnic_tx_queues=0,0,2
```

Note: The interface configuration settings must be identical on the two sides.

These configuration settings give you 3 software-defined NIC interfaces on the TILEncore-Gx card, named `gxpci0-0`, `gxpci0-1` and `gxpci0-2`.

7 Improving Performance on a TILE-Gx Processor

In order to avoid a CPU bottleneck under heavy network traffic, interrupt migration can be used to move interrupts away from heavily-loaded host processors. Examples of this load balancing could be to configure two busy virtual NIC interfaces to interrupt separate processors, or to schedule NIC interrupts away from a processor which is busy with unrelated application processing. The TILE-Gx host PCIe driver supports PCI MSI-X for host interrupts, providing a unique interrupt vector to each RX/TX queue pair. For detailed instructions, refer to the Linux documentation Documentation/`IRQ-affinity.txt`.

8 Performance Measurements and Results

This section presents some performance measurement results using two types of applications over a PCIe Gen2 x8 link between a TILEncore-Gx card with the TILE-Gx36 running at 1GHz and a host with Intel Xeon™ processors.

Running the example program in `$TILERA_ROOT/examples/trio/gxpci_host_nic` using a single software-defined NIC interface (by defining `nic_ports=1` on the host side, and `pcie_host_nic_ports=0,0,1` on the Tile-side), we obtained the following performance:

Table 1. Performance Data for Unidirection Throughput

Packet Size (Bytes)	Unidirectional Throughput (Gbps)
64	0.8
512	6.63
1024	13.87
1500	20.03
2048	27.33
4096	28.50

In the host kernel IP forwarding test which uses the Software-defined NIC configuration, the packets are transmitted from an external traffic generator to one or more of the TILE-Gx's XAUI ports and transferred to the x86 host via the PCIe. The host kernel IP stack performs the IP forwarding and routes the packets back to the TILE-Gx and out of its XAUI ports.

Table 2. Performance Data for Varying Port and Queue Configurations

Configuration	64-Byte traffic (Mbps)	64-Byte traffic (Gbps)	1500-Byte traffic (Mbps)	1500-Byte traffic (Gbps)	256-Byte traffic (Mbps)	256-Byte traffic (Gbps)
2 Software-defined NICs 1 RX /1 TX queue	1.48	1.00	1.28	15.61	N/A	N/A
2 Software-defined NICs 2 RX /2 TX queues	2.49	1.67	1.63	19.83	N/A	N/A
4 Software-defined NICs 1 RX /1 TX queue	2.46	1.65	1.59	19.38	N/A	N/A
4 Software-defined NICs 2 RX /2 TX queues	3.15	2.12	1.98	24.08	3.16	6.99